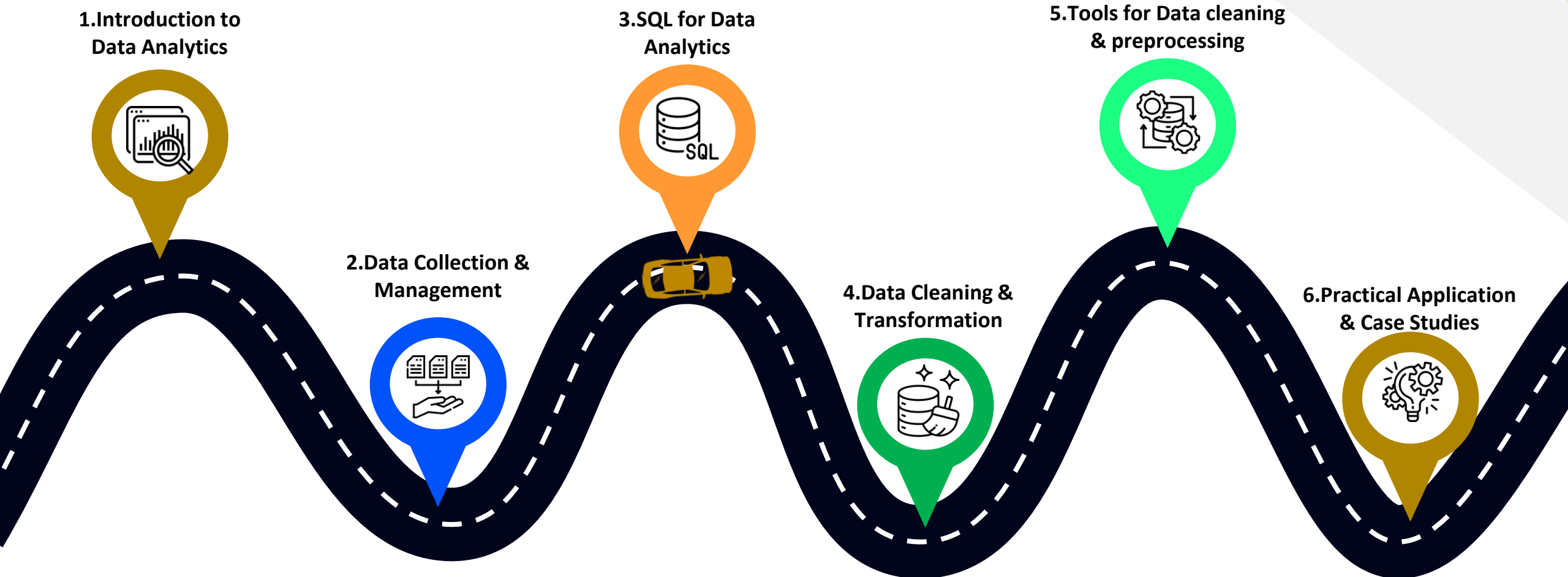




Part Four: SQL for Data Analytics

Course Road Map

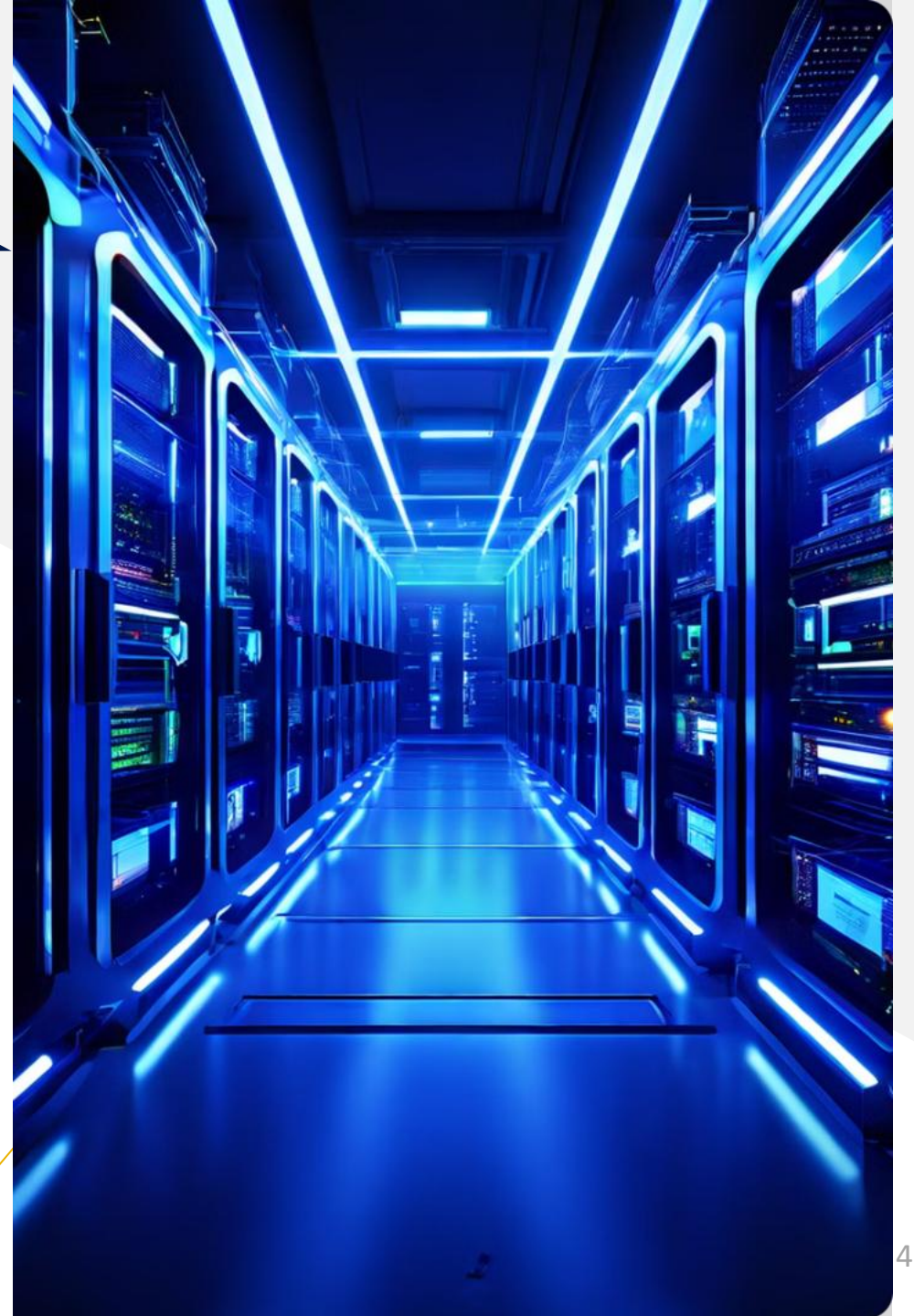


Agenda

- 1 **Introduction to SQL**
- 2 **DQL Command**
SELECT
- 3 **DML Commands**
INSERT, UPDATE and DELETE
- 4 **DDL Commands**
CREATE, DROP, ALTER, TRUNCATE, COMMENT, RENAME ...etc
- 5 **DCL Commands**
GRANT and REVOKE
- 6 **TCL Commands**
BEGIN TRANSACTION, COMMIT, ROLLBACK and SAVEPOINT

SQL Queries

SQL, or Structured Query Language, is a powerful programming language used to manage and manipulate relational databases. SQL queries allow you to perform a wide range of operations, from retrieving data to modifying database structures.



Introduction to SQL

What is SQL?

SQL is a language used to communicate with and manage relational databases. It allows users to create, read, update, and delete data stored in database tables.

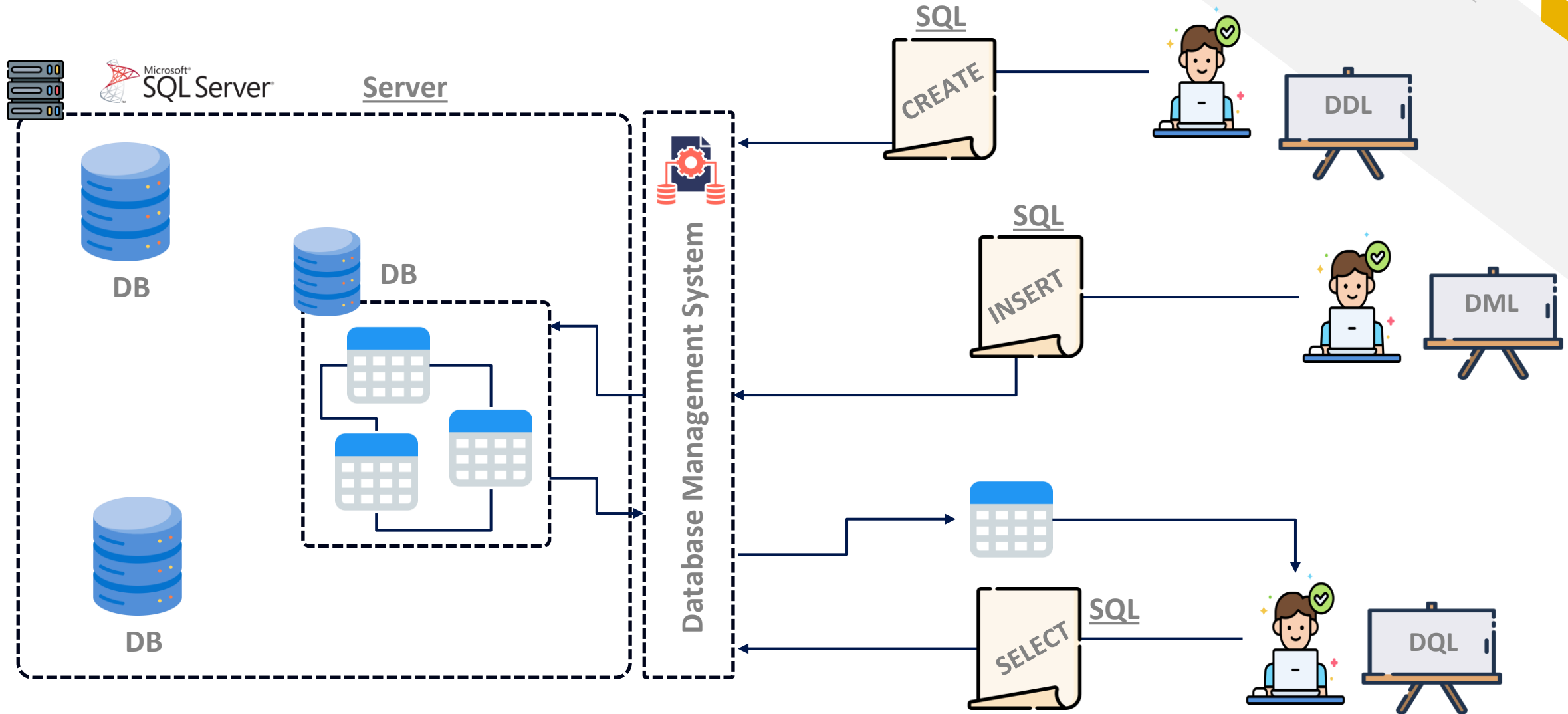
Why Use SQL?

SQL is a standard language that provides a consistent and efficient way to interact with databases. It enables developers and analysts to access, manipulate, and manage data quickly and effectively.

SQL Applications

SQL is used in a variety of applications, including e-commerce platforms, social media networks, financial systems, and more. It is an essential skill for data analysts, database administrators, and software engineers.

SQL



Basic SQL Syntax

1 SQL Statements

The main SQL statements include SELECT, INSERT, UPDATE, and DELETE, which allow you to retrieve, add, modify, and remove data from a database.

3 SQL Syntax Structure

SQL statements follow a specific syntax structure, with keywords, identifiers, and operators arranged in a specific order to perform desired operations.

2 SQL Clauses

Clauses like WHERE, ORDER BY, and GROUP BY are used to filter, sort, and aggregate data in SQL queries.

4 SQL Formatting

Proper formatting, including indentation and capitalization, makes SQL queries more readable and maintainable for developers and analysts.

SQL Data Types

1

Numeric

Integers, floating-point, and decimal numbers for storing numerical data.

3

Date and Time

Specialized data types for storing and manipulating date, time, and timestamp information.

2

Text

Character strings and text-based data, such as names, addresses, and descriptions.

4

Boolean

Logical values, such as true or false, used for storing binary information.



SQL Commands

DDL, DML, DCL, TCL, and DQL

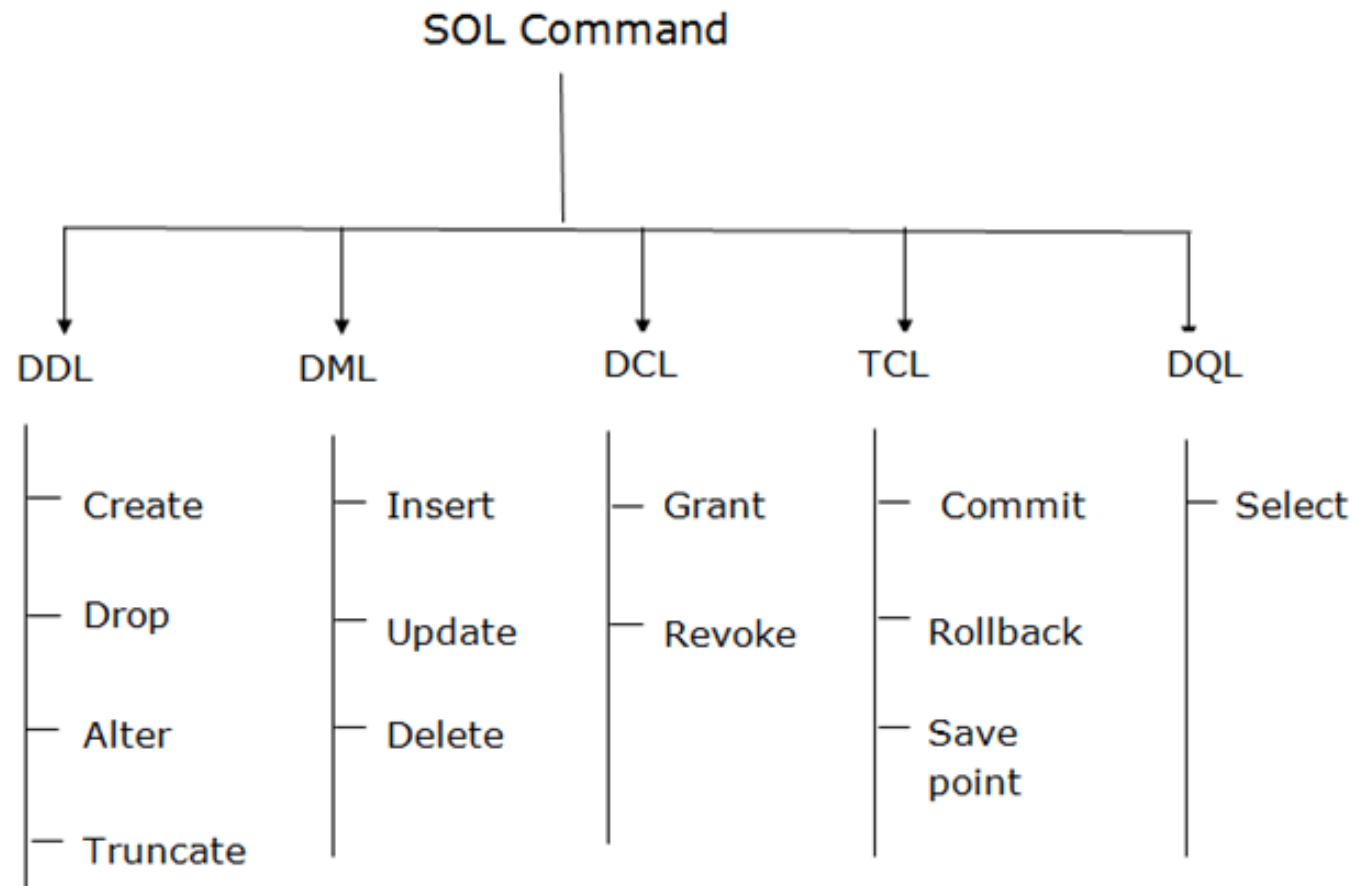
SQL Commands

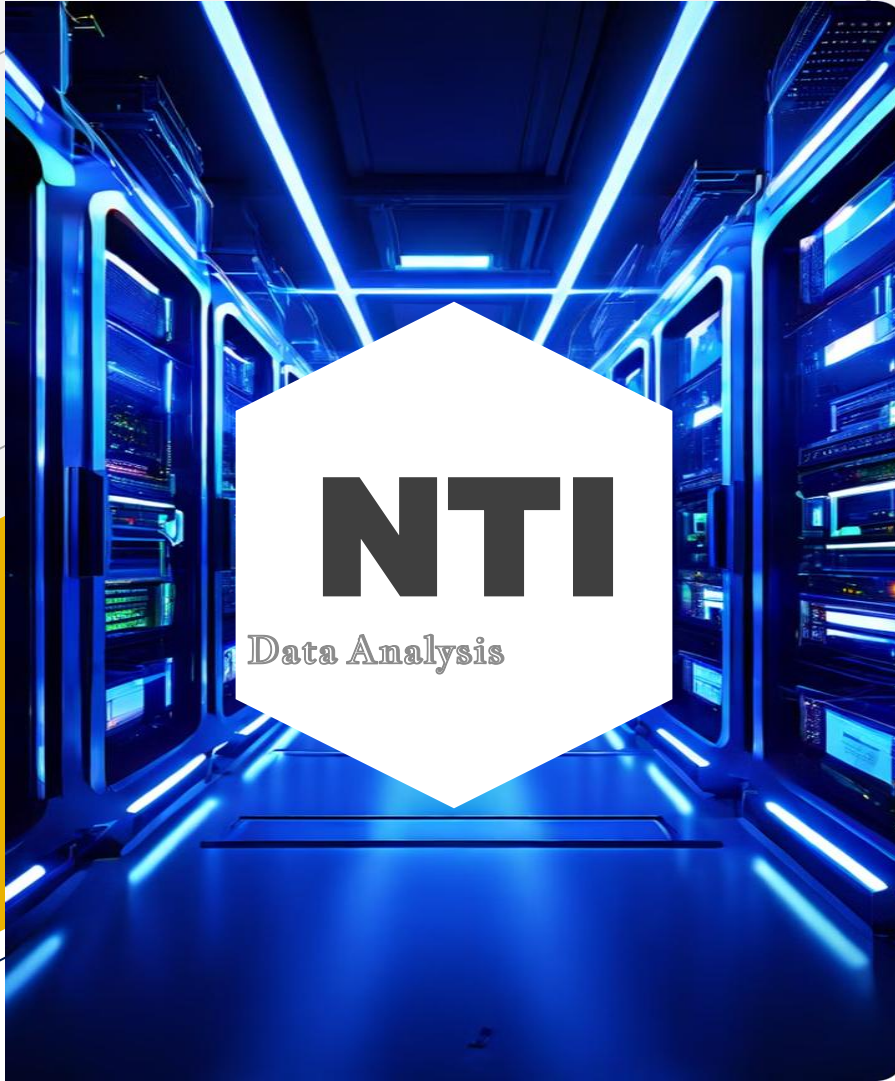
- SQL commands are instructions. It is used to communicate with the database. It is also used to perform specific tasks, functions, and queries of data.
- SQL can perform various tasks like create a table, add data to tables, drop the table, modify the table, set permission for users.



Types of SQL Commands

- There are five types of SQL commands: DDL, DML, DCL, TCL, and DQL.





DQL Command

SELECT

DQL (Data Query Language)

- **DQL** statements are used for performing queries on the data within schema objects (the database). It includes the **SELECT** statement.
- **SELECT** command allows getting the data out of the database to perform operations with it.



DQL Command

There is only one DQL command in SQL i.e.

Command	Description	Syntax
SELECT	It is used to retrieve data from the database	<pre>SELECT column1, column2, ... FROM table_name WHERE condition;</pre>

SELECT Statements

SELECT Basics

The SELECT statement is used to retrieve data from a database. It allows you to specify which columns you want to include in the result set.

1

2

Column Selection

You can select specific columns or use the wildcard (*) to retrieve all columns from a table.

Aliases and Expressions

You can use aliases to rename columns and create calculated expressions to perform operations on the data.

3

4

DISTINCT Column Selection

SELECT DISTINCT statement is used to return only distinct (different) values.

WHERE Clauses

Filtering Data

The WHERE clause allows you to filter the data returned by a SELECT statement based on specific conditions.

Comparison Operators

You can use comparison operators like =, >, <, >=, <=, and != to define the conditions for filtering data.

Logical Operators

Logical operators like AND, OR, and NOT can be used to combine multiple conditions in the WHERE clause.

Pattern Matching

The LIKE operator, combined with wildcard characters, allows you to perform pattern matching on text data.

Subqueries

Use Case	Explanation
Nested Queries	Subqueries can be nested inside other SQL statements, allowing you to perform complex data analysis.
Correlated Subqueries	Correlated subqueries reference a column from the outer query, enabling you to perform more sophisticated data filtering and retrieval.
Scalar Subqueries	Scalar subqueries return a single value, which can be used in the SELECT, WHERE, or HAVING clauses of an outer query.
Subquery Expressions	Subqueries can be used in conjunction with set operators like IN, EXISTS, and NOT EXISTS to create powerful query expressions.

LIKE Operator

- The LIKE operator is used in a WHERE clause to search for a specified pattern in a column.
- There are two wildcards often used in conjunction with the LIKE operator:
 - The percent sign % represents zero, one, or multiple characters
 - The underscore sign _ represents one, single character
 - [] Represents any single character within the brackets
 - ^ Represents any character not in the brackets
 - - Represents any single character within the specified range
- Syntax:
 - `SELECT column1, column2, ...`
`FROM table_name`
`WHERE columnN LIKE pattern;`

IN Operator

- Syntax
 - `SELECT column_name(s)`
`FROM table_name`
`WHERE column_name IN (value1, value2, ...);`

BETWEEN Operator

- Syntax
 - `SELECT column_name(s)`
`FROM table_name`
`WHERE column_name BETWEEN value1 AND value2;`
- **NOT BETWEEN**

SQL Aliases

- Syntax:
- When alias is used on column:
 - `SELECT column_name AS alias_name`
`FROM table_name;`
- When alias is used on table:
 - `SELECT column_name(s)`
`FROM table_name AS alias_name;`

Joins



Inner Join

Combines rows from two tables based on a related column, returning only the matching rows.



Outer Join

Includes all rows from one table and the matching rows from another table, filling in missing values with NULL.



Left/Right Join

Returns all rows from the left/right table and the matching rows from the right/left table.

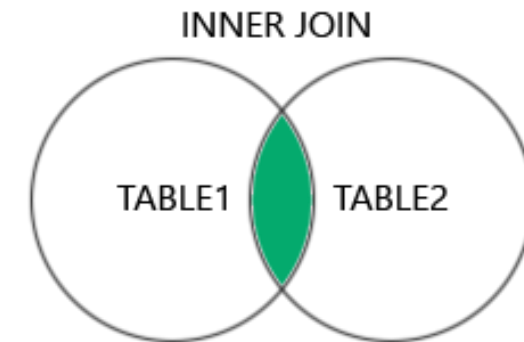


Cross Join

Generates a Cartesian product, combining every row from one table with every row from another table.

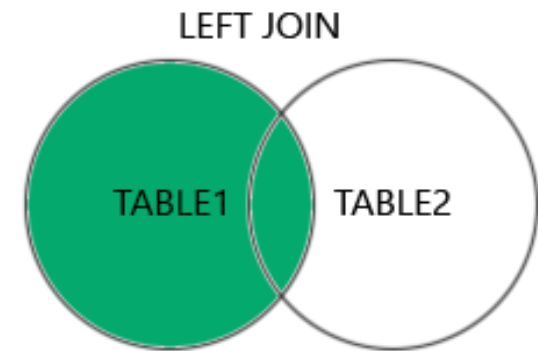
INNER JOIN

- The INNER JOIN keyword selects records that have matching values in both tables.
- Syntax:
 - `SELECT column_name(s)`
`FROM table1`
`INNER JOIN table2`
`ON table1.column_name = table2.column_name;`
- JOIN or INNER JOIN return the same result.



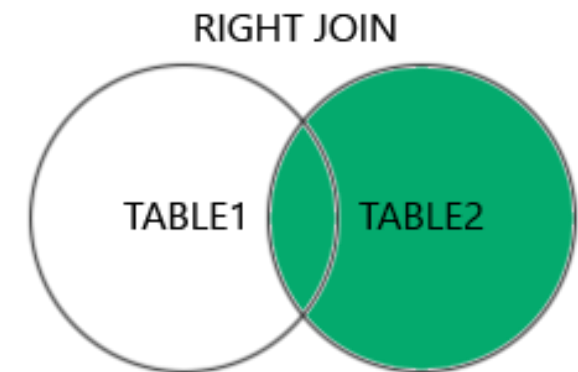
LEFT JOIN

- Syntax:
 - `SELECT column_name(s)`
`FROM table1`
`LEFT JOIN table2`
`ON table1.column_name = table2.column_name;`
- In some databases LEFT JOIN is called LEFT OUTER JOIN.



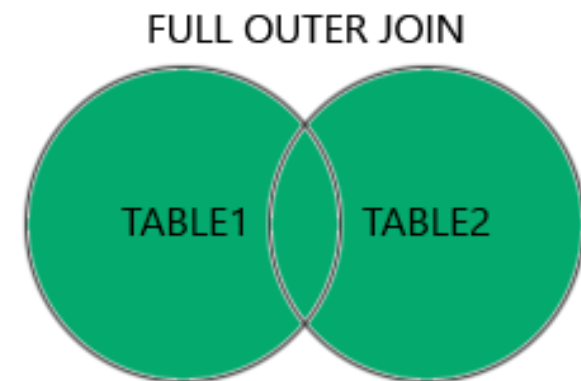
RIGHT JOIN

- Syntax:
 - `SELECT column_name(s)`
`FROM table1`
`RIGHT JOIN table2`
`ON table1.column_name = table2.column_name;`
- In some databases RIGHT JOIN is called RIGHT OUTER JOIN.



FULL OUTER JOIN

- Syntax:
 - `SELECT column_name(s)`
`FROM table1`
`FULL OUTER JOIN table2`
`ON table1.column_name = table2.column_name`
`WHERE condition;`
- FULL OUTER JOIN and FULL JOIN are the same.



Self Join

- A self join is a regular join, but the table is joined with itself.
- Syntax:
 - `SELECT column_name(s)`
`FROM table1 T1, table1 T2`
`WHERE condition;`
 - *T1* and *T2* are different table aliases for the same table.

Aggregation Functions

1

SUM

Calculates the total of a numeric column.

2

AVG

Calculates the average value of a numeric column.

3

COUNT

Counts the number of rows in a table or a specific column.

4

MIN/MAX

Returns the minimum or maximum value in a column.

Aggregate Functions

- Aggregate Functions Syntax:
 - `SELECT MIN | MAX | SUM | AVG | COUNT (column_name)
FROM table_name
WHERE condition;`
- If you specify a column name instead of (*) with Count, NULL values will not be counted.
- You can ignore duplicates by using the DISTINCT keyword in the COUNT() function. Syntax:
 - `SELECT COUNT(DISTINCT column_name)
FROM table_name;`
- In the AVG() function NULL values are ignored.
- The parameter inside the SUM() function can also be an expression like `Quantity * 10`.

GROUP BY Statement

- The GROUP BY statement groups rows that have the same values into summary rows, like "find the number of customers in each country".
- The GROUP BY statement is often used with aggregate functions.
- GROUP BY Syntax:
 - `SELECT column_name(s)`
`FROM table_name`
`WHERE condition`
`GROUP BY column_name(s)`
`ORDER BY column_name(s);`

HAVING Clause

- The HAVING clause was added to SQL because the WHERE keyword cannot be used with aggregate functions.
- HAVING Syntax:
 - `SELECT column_name(s)`
`FROM table_name`
`WHERE condition`
`GROUP BY column_name(s)`
`HAVING condition`
`ORDER BY column_name(s);`

UNION & UNION ALL Operator

- The UNION operator is used to combine the result-set of two or more SELECT statements.
- UNION Syntax
 - `SELECT column_name(s) FROM table1`
`UNION`
`SELECT column_name(s) FROM table2;`
- The UNION operator selects only distinct values by default. To allow duplicate values, use UNION ALL.
- UNION ALL Syntax
 - `SELECT column_name(s) FROM table1`
`UNION ALL`
`SELECT column_name(s) FROM table2;`

EXISTS Operator

- The EXISTS operator is used to test for the existence of any record in a subquery.
- EXISTS Syntax:
 - `SELECT column_name(s)`
`FROM table_name`
`WHERE EXISTS`
`(SELECT column_name FROM table_name WHERE condition);`

ANY and ALL Operators

- The ANY and ALL operators allow you to perform a comparison between a single column value and a range of other values.
- ANY Syntax:
 - ```
SELECT column_name(s)
FROM table_name
WHERE column_name operator ANY
 (SELECT column_name
FROM table_name
WHERE condition);
```

# ANY and ALL Operators

- ALL means that the condition will be true only if the operation is true for all values in the range.
- ALL Syntax:
  - ```
SELECT column_name(s)
FROM table_name
WHERE column_name operator ALL
      (SELECT column_name
FROM table_name
WHERE condition);
```

What is a NULL Value?

- A field with a NULL value is a field with no value.
- If a field in a table is optional, it is possible to insert a new record or update a record without adding a value to this field. Then, the field will be saved with a NULL value.



How to Test for NULL Values?

- IS NULL

- The IS NULL operator is used to test for empty values (NULL values).
- IS NULL Syntax:
 - SELECT column_names
FROM table_name
WHERE column_name IS NULL;

- IS NOT NULL

- The IS NOT NULL operator is used to test for non-empty values (NOT NULL values).
- IS NOT NULL Syntax:
 - SELECT *column_names*
FROM *table_name*
WHERE *column_name* IS NOT NULL;

SQL NULL Functions

- **ISNULL()** function lets you return an alternative value when an expression is NULL:
 - SELECT ProductName, UnitPrice * (UnitsInStock + ISNULL(UnitsOnOrder, 0))
FROM Products;**SQL**
- or we can use the **COALESCE()** function, like this:
 - SELECT ProductName, UnitPrice * (UnitsInStock + COALESCE(UnitsOnOrder, 0))
FROM Products;

The SQL SELECT TOP Clause:

- The **SELECT TOP** clause is used to specify the number of records to return.
- **SELECT TOP Syntax:**
 - `SELECT TOP number|percent column_name(s)
FROM table_name
WHERE condition;`
- **SQL TOP PERCENT:** SQL statement selects the first 50% of the records from the table
- **SELECT TOP PERCENT Syntax:**
 - `SELECT TOP 50 PERCENT *
FROM table_name WHERE condition;`

CASE Expression

- The CASE expression goes through conditions and returns a value when the first condition is met (like an if-then-else statement). If no conditions are true, it returns the value in the ELSE clause.
- If there is no ELSE part and no conditions are true, it returns NULL.
- CASE Syntax:
 - CASE
 WHEN *condition1* THEN *result1*
 WHEN *condition2* THEN *result2*
 WHEN *conditionN* THEN *resultN*
 ELSE *result*
END;



DML Commands

INSERT, UPDATE and DELETE

DML (Data Manipulation Language)

- **DML** stands for Data Manipulation Language. It deals with data manipulation and includes the most common SQL statements such as INSERT, UPDATE, DELETE, etc.
- **DML** statements are not auto-committed, meaning the changes can be rolled back if necessary.



List of DML Commands

Some DML commands and their syntax are:

Command	Description	Syntax
INSERT	Insert data into a table	INSERT INTO table_name (column1, column2, ...) VALUES (value1, value2, ...);
UPDATE	Update existing data within a table	UPDATE table_name SET column1 = value1, column2 = value2 WHERE condition;
DELETE	Delete records from a database table	DELETE FROM table_name WHERE condition;

Important Notes:

- the **WHERE** clause in the **UPDATE** statement specifies which record(s) that should be updated. If you omit the **WHERE** clause, all records in the table will be updated!
- Notice the **WHERE** clause in the **DELETE** statement specifies which record(s) should be deleted. If you omit the **WHERE** clause, all records in the table will be deleted!
- It is possible to delete all rows in a table without deleting the table. This means that the table structure, attributes, and indexes will be intact:
 - `DELETE FROM table_name;`

SELECT INTO Statement

- The SELECT INTO statement copies data from one table into a new table.
- SELECT INTO Syntax:
 - Copy all columns into a new table:
 - `SELECT *`
`INTO newtable [IN externaldb]`
`FROM oldtable`
`WHERE condition;`
 - Copy only some columns into a new table:
 - `SELECT column1, column2, column3, ...`
`INTO newtable [IN externaldb]`
`FROM oldtable`
`WHERE condition;`
- The new table will be created with the column-names and types as defined in the old table. You can create new column names using the AS clause.

INSERT INTO SELECT

- The INSERT INTO SELECT statement copies data from one table and inserts it into another table
- The INSERT INTO SELECT statement requires that the data types in source and target tables match.
- Note: The existing records in the target table are unaffected.
- INSERT INTO SELECT Syntax
 - Copy all columns from one table to another table:
 - INSERT INTO *table2*
SELECT * FROM *table1*
WHERE *condition*;
 - Copy only some columns from one table into another table:
 - INSERT INTO *table2* (*column1, column2, column3, ...*)
SELECT *column1, column2, column3, ...*
FROM *table1*
WHERE *condition*;



DDL Commands

CREATE, DROP, ALTER, TRUNCATE,
COMMENT, RENAME ...etc

DDL (Data Definition Language)

- It simply deals with descriptions of the database schema and is used to create and modify the structure of database objects in the database.
- DDL is a set of SQL commands used to create, modify, and delete database structures but not data.



List of DDL Commands

Some DDL commands and their syntax are:

Command	Description	Syntax
CREATE	Create database or its objects (table, index, function, views, store procedure, and triggers)	CREATE TABLE table_name (column1 data_type, column2 data_type, ...);
DROP	Delete objects from the database	DROP TABLE table_name; DROP DATABASE database_name;
ALTER	Alter the structure of the database	ALTER TABLE table_name ADD COLUMN column_name data_type;
TRUNCATE	Remove all records from a table, including all spaces allocated for the records are removed	TRUNCATE TABLE table_name;
COMMENT	Add comments to the data dictionary	COMMENT 'comment_text' ON TABLE table_name;
RENAME	Rename an object existing in the database	RENAME TABLE old_table_name TO new_table_name;

List of DDL Commands

Some DDL commands and their syntax are:

Command	Description	Syntax
CREATE INDEX	Creates an index on a table to improve a specific query performance.	CREATE INDEX <i>index_name</i> ON <i>table_name</i> (<i>column1</i> , <i>column2</i> , ...);
CREATE VIEW	Creates a view, a virtual table based on one or more existing tables.	CREATE VIEW <i>view_name</i> AS <i>select_statement</i>
CREATE PROCEDURE	Creates a stored procedure, a precompiled SQL statement that can be run multiple times with different parameters.	CREATE PROCEDURE <i>procedure_name</i> AS <i>sql_statement</i> ;
CREATE FUNCTION	Creates a custom user-defined function that can be utilized in SQL statements.	CREATE FUNCTION <i>function_name</i> ... ;
CREATE TRIGGER	Creates a trigger, a type of stored procedure that is automatically executed when certain events occur, such as inserting, updating, or deleting data in a table.	CREATE TRIGGER <i>trigger-name</i> ON <i>table-name</i> ... ;

BACKUP DATABASE

- The BACKUP DATABASE statement is used in SQL Server to create a full back up of an existing SQL database.
- **Syntax:**
 - BACKUP DATABASE *databasename*
TO DISK = '*filepath*';
- The SQL **BACKUP WITH DIFFERENTIAL** Statement
 - A differential back up only backs up the parts of the database that have changed since the last full database backup.
 - **Syntax:**
 - BACKUP DATABASE *databasename*
TO DISK = '*filepath*'
WITH DIFFERENTIAL;

What is a Stored Procedure?

- A stored procedure is a prepared SQL code that you can save, so the code can be reused over and over again.
- You can also pass parameters to a stored procedure, so that the stored procedure can act based on the parameter value(s) that is passed.
- Stored Procedure Syntax
 - CREATE PROCEDURE *procedure_name* @*parameter_name* *parameter_datatype*
AS
sql_statement
GO;
- Execute a Stored Procedure
 - EXEC *procedure_name* @*parameter_name* = *value* ;



DCL Commands

GRANT and REVOKE

DCL (Data Control Language)

- DCL includes commands such as GRANT and REVOKE which mainly deal with the rights, permissions, and other controls of the database system.



List of DCL Commands

Some DCL commands and their syntax are:

Command	Description	Syntax
GRANT	Assigns new privileges to a user account, allowing access to specific database objects, actions, or functions.	GRANT privilege_type [(column_list)] ON [object_type] object_name TO user [WITH GRANT OPTION];
REVOKE	Removes previously granted privileges from a user account, taking away their access to certain database objects or actions.	REVOKE [GRANT OPTION FOR] privilege_type [(column_list)] ON [object_type] object_name FROM user [CASCADE];



TCL Commands

BEGIN TRANSACTION, COMMIT, ROLLBACK
and SAVEPOINT

TCL (Transaction Control Language)

- Transactions group a set of tasks into a single execution unit. Each transaction begins with a specific task and ends when all the tasks in the group are successfully completed. If any of the tasks fail, the transaction fails.



List of TCL Commands

Some TCL commands and their syntax are:

Command	Description	Syntax
BEGIN TRANSACTION	Starts a new transaction	BEGIN TRANSACTION [transaction_name];
COMMIT	Saves all changes made during the transaction	COMMIT;
ROLLBACK	Undoes all changes made during the transaction	ROLLBACK;
SAVEPOINT	Creates a savepoint within the current transaction	SAVEPOINT savepoint_name;

SQL Comments

- Comments are used to explain sections of SQL statements, or to prevent execution of SQL statements.
- Single Line Comments
 - Single line comments start with --.
 - Any text between -- and the end of the line will be ignored (will not be executed).
- Multi-line Comments
 - Multi-line comments start with /* and end with */.
 - Any text between /* and */ will be ignored.



Thank You.



Ola El-Shiekh



01021183564



ola.elshiekh71@gmail.com



<https://www.linkedin.com/in/ola-el-shiekh/>